
EDGE COMPUTING PARA CONTROL BIÓNICO CON SEÑALES CEREBRALES

David Tapia Pino
david.tapiap@sansano.usm.cl

Julio 2025

1 Introducción

En los últimos años, el auge del aprendizaje automático y la creciente capacidad de cómputo en dispositivos embebidos han impulsado el desarrollo de técnicas que permiten realizar inferencia directamente en sistemas con recursos limitados, esta familia de algoritmos es conocida como Tiny Machine Learning (TinyML). Al ser combinado con Edge Computing, paradigma que promueve la ejecución de procesos computacionales cercanos al punto donde se generan los datos, es posible reducir tanto la latencia como el tráfico de red, además de mejorar la autonomía en sistemas críticos. Herramientas como TensorFlow Lite y STM32Cube.AI facilitan la implementación de estos modelos optimizados en hardware específico, permitiendo aplicaciones avanzadas en contextos donde la rapidez y eficiencia energética son fundamentales.

En este contexto, el presente trabajo aborda el desafío de clasificar señales cerebrales obtenidas mediante electrocorticografía (ECoG) para predecir movimientos de flexión de dedos, utilizando el Dataset 4 de la reconocida competencia BCI Competition IV. Se propone un enfoque basado en redes neuronales profundas, cuidadosamente adaptado y optimizado para ser implementado en un microcontrolador STM32 NUCLEO-L476RG, como prototipo orientado al control de interfaces neuronales para aplicaciones como prótesis robóticas. Debido a restricciones inherentes del hardware utilizado, se exploran y discuten diferentes técnicas para adaptar estos modelos, incluyendo cuantización, destilación de conocimiento y optimizaciones específicas en la arquitectura, buscando equilibrar el desempeño predictivo con las limitaciones computacionales existentes.

Los códigos asociados a este proyecto se encuentran en un [repositorio público en Github](#).

2 Marco teórico

2.1 Edge Computing

Edge Computing, o computación en el borde (también conocida como computación de frontera), es un paradigma de computación de datos distribuidos en el cual parte de los cálculos y del procesamiento de los datos se realiza en “el borde” de la red de computadores. A diferencia de lo que ocurre con el *Cloud Computing*, en este paradigma el procesamiento de los datos se efectúa “más cerca” del punto en que estos son generados.

Para ilustrar esto mediante un ejemplo, si se dispone de un sistema de video vigilancia con cámaras IP, una alternativa basada en Cloud Computing consistiría en enviar el stream de video 24/7 a un servidor remoto, donde se ejecutarían algoritmos de detección de personas o robos. En cambio, una aproximación utilizando Edge Computing implicaría dar a la cámara la capacidad, local, de analizar las imágenes usando algoritmos sencillos de IA para detectar movimiento o anomalías. Luego la cámara solo envía el stream de video a un servidor remoto para detectar personas o robos con algoritmos más avanzados cuando ha detectado localmente que ha ocurrido algo inusual, para disminuir tanto como sea posible el tráfico de red asociado a enviar datos de video al servidor en la nube. Como parte del procesamiento ocurre en el mismo dispositivo donde se generan los datos, en este caso, la cámara, esta estrategia se clasifica como

Cloud Computing vs Edge Computing

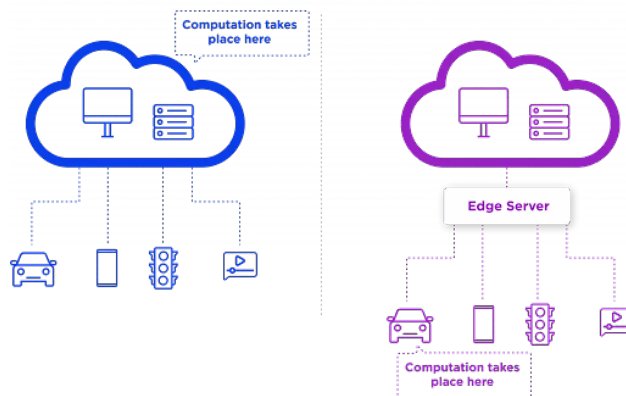


Figure 1: Diagrama que muestra las diferencias entre Edge y Cloud Computing. Extraído de [2].

Edge Computing.

Existe además un paradigma intermedio entre el borde (*Edge*) y la nube (*Cloud*), denominado *Fog Computing* o computación “en la niebla”. Aplicar *Fog Computing* en este contexto implicaría que las cámaras envíen sus datos de video a un servidor local en el recinto o a un dispositivo de borde más potente, el cual realizaría el análisis de detección de personas o robos. Solo si dicho servidor local confirma un evento sospechoso, se enviaría la información procesada hacia un servidor remoto en la nube para almacenamiento o acciones posteriores. De este modo, se aprovecha una infraestructura intermedia entre los dispositivos de borde y la nube, reduciendo la latencia y el tráfico de red.

La principal razón por la que estos paradigmas existen es para tener la capacidad de manejar el trade-off entre el poder de cómputo y el tiempo de respuesta. Mientras que la nube ofrece el mayor poder de cómputo de las tres alternativas, ejecutar procesamiento en el borde de la red resulta significativamente más rápido, ya que se realiza en el mismo lugar donde se generan los datos, con el riesgo de que el cómputo no puede ser tan avanzado o preciso como lo sería en la nube.

2.2 TinyML

TinyML es una rama del *Machine Learning* dedicada a la construcción de modelos extremadamente ligeros, diseñados para realizar inferencia en hardware con capacidades limitadas, ya sea por restricciones de memoria, bajo poder de cómputo o limitaciones energéticas.

En el ciclo de vida de los modelos de aprendizaje automático existen dos etapas fundamentales: el entrenamiento y la inferencia. Durante el entrenamiento, el modelo ajusta sus parámetros internos a partir de los datos disponibles, aprendiendo patrones o características relevantes. En la etapa de inferencia, se presentan al modelo datos que no había visto previamente, para que este realice predicciones o clasificaciones basándose en lo aprendido durante el entrenamiento. Por ejemplo, un modelo de clasificación de imágenes que distingue entre perros y gatos tendría una fase de entrenamiento en la que observaría numerosas imágenes de ambos animales, recibiendo etiquetas explícitas que le dicen a qué categoría pertenece cada imagen (este es un ejemplo de “aprendizaje supervisado”). Posteriormente, en la fase de inferencia, se le presentaría una imagen nueva, que no ha sido vista durante el entrenamiento, y el modelo debería predecir si corresponde a un perro o a un gato. Aunque la inferencia suele requerir menos recursos computacionales que el entrenamiento, el tamaño y complejidad del modelo original pueden hacer que esta etapa resulte aún demasiado exigente para sistemas embebidos con memoria limitada o bajo poder de procesamiento.

El flujo de trabajo típico en *TinyML* consiste en realizar el entrenamiento del modelo en máquinas de alta capacidad, como estaciones de trabajo con GPU o incluso utilizando servidores remotos, y posteriormente optimizar el modelo mediante técnicas como cuantización, poda o destilación. El modelo resultante puede entonces ejecutarse eficiente-

mente en dispositivos de recursos reducidos, permitiendo realizar inferencia de manera autónoma y local sin necesidad de conexión constante a servidores potentes. A continuación se describen algunas de las técnicas utilizadas en TinyML para reducir el tamaño y la complejidad de las redes neuronales:

- Cuantización:** Los pesos de las redes neuronales, por defecto, suelen ser representados como números de punto flotante, utilizando habitualmente entre 4 y 8 bytes por peso (32 o 64 bits). La cuantización consiste en reducir la precisión numérica de estos pesos, convirtiéndolos a representaciones de menor tamaño, como números de 8 bits (`int8`) o incluso 4 bits en algunos casos. Esto disminuye el tamaño total del modelo y permite inferencias más rápidas y eficientes en dispositivos de recursos limitados [9]. Existe también la cuantización binaria, donde los pesos toman valores de sólo 1 bit. Sin embargo, esta técnica extrema requiere entrenar redes diseñadas específicamente para funcionar con representaciones binarias, y no es la forma más común de cuantización en TinyML debido a las pérdidas de precisión que puede implicar [4]. Las técnicas de cuantización pueden ser clasificadas también según el momento en que se realiza la cuantización, teniéndose así el enfoque PTQ (*Post-Training Quantization*), en que se tiene un modelo entrenado con precisión completa al que se le aplica una cuantización de sus pesos, y por otro lado está el enfoque QAT (*Quantization-Aware-Training*), en el que, si bien el entrenamiento sigue haciéndose con precisión completa, se agregan capas de cuantización entre las neuronas, que simulan cómo sería el funcionamiento de la red neuronal si sus pesos estuviesen cuantizados, lo que hace que la red llegue a una representación de 32 (o 64) bits que funcione satisfactoriamente cuando se aplique la cuantización deseada. QAT suele ser necesario para cuantizaciones más estrictas, con pesos de 1 a 4 bits, mientras que PTQ suele ser suficiente para cuantizaciones a 8 bits [9].
- Poda (Pruning):** La poda consiste en eliminar conexiones o neuronas que tienen un impacto mínimo en la salida de la red. Una forma de lograr esto es identificando los pesos cuyo valor absoluto es muy pequeño y forzándolos a cero, reduciendo así la cantidad de parámetros activos en el modelo. Otra forma de identificar los pesos a eliminar considera el producto entre el peso y la derivada de la función de pérdida con respecto a este peso, así no solo se considera la magnitud del peso, si no también qué tanto afecta en el proceso de aprendizaje utilizando la expansión de Taylor de primer o hasta según orden de la función de pérdida [8]. Si bien la poda puede disminuir la cantidad de cálculos necesarios y ahorrar memoria, es importante tener en cuenta que una poda excesiva puede afectar la precisión de la inferencia. Además, si no se realiza correctamente, podría aumentar el riesgo de sobreajuste (*overfitting*) al eliminar redundancias que ayudan a que la red neuronal pueda generalizar bien y tener un buen desempeño en la etapa de inferencia con datos nuevos [14].
- Destilación del conocimiento (Knowledge Distillation):** Esta técnica consiste en entrenar un modelo más pequeño, denominado “alumno” (*student*), para imitar el comportamiento de un modelo más grande y preciso, denominado “profesor” (*teacher*). Durante el entrenamiento, el modelo alumno no aprende solo de los datos originales, sino también de las salidas “suavizadas” generadas por el modelo profesor. Esta suavización se refiere a que la función que se encarga de decidir el valor de la salida de la red neuronal (la función softmax para los modelos de clasificación) ya no está “tan segura” como lo estaba originalmente, pues se ha suavizado la distribución de las probabilidades de las salidas, por ejemplo, se puede tener entonces que el profesor diga que está un 60% seguro que la imagen a la entrada es un gato, un 30% de que es un zorro y porcentajes menores para las otras clases, mientras que originalmente el modelo profesor podría haber dicho que estaba seguro en un 90% que se trataba de un gato y porcentajes pequeños para todas las otras clases. Esta “indecisión” permite que el modelo alumno sea capaz de captar información sobre la distribución relativa de las clases y las similitudes entre estas, aprendiendo no solo a imitar al profesor, si no también a imitar la forma en que este estructura su conocimiento, logrando un entendimiento más profundo de la distribución subyacente. La destilación permite transferir la capacidad de generalización de un modelo complejo a uno mucho más compacto, logrando mantener un buen desempeño en tareas de inferencia con un tamaño de modelo adecuado para sistemas embebidos [3].

3 Estado del Arte

3.1 BCI Competition

Berlin Brain-Computer Interface es una iniciativa de investigación liderada por la Universidad Técnica de Berlín (TU Berlin) que ha contribuido significativamente al desarrollo de interfaces cerebro-computador (BCI, por sus siglas en inglés). Entre sus aportes más relevantes se encuentran las *BCI Competitions*, organizadas con el objetivo de fomentar la innovación y comparar de manera estandarizada distintas técnicas de procesamiento de señales cerebrales y de clasificación de estados mentales [1]. Estas competencias proporcionaban conjuntos de datos desafiantes, recogidos bajo condiciones controladas, para que investigadores de todo el mundo propusieran soluciones que permitieran interpretar las señales cerebrales de manera eficiente y precisa. Una de las ediciones más reconocidas fue la *BCI Competition IV* organizada en 2008. Esta competencia presentó una serie de desafíos que abarcaban diferentes modalidades de adquisición de señales cerebrales, incluyendo electroencefalografía (EEG) y electrocorticografía (ECoG).

El conjunto de datos utilizado en este proyecto corresponde al **Dataset 4** de la BCI Competition IV [7]. Este dataset contiene registros de señales ECoG obtenidos de tres sujetos, utilizando entre 48 y 65 electrodos implantados sobre la superficie de la corteza cerebral de cada participante. Además de las señales cerebrales, el conjunto incluye información de un guante con sensores que miden la posición de cada dedo de la mano en tiempo real, como se muestra en la Figura 2

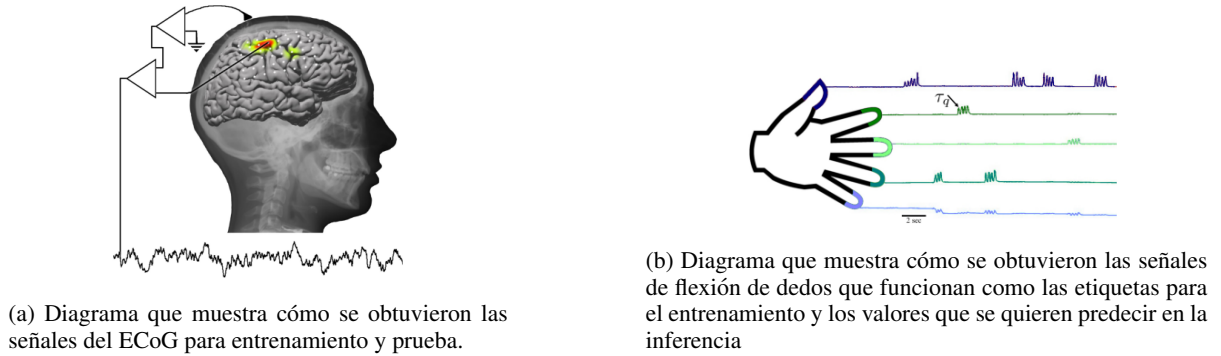


Figure 2: Diagramas que explican la construcción del Dataset4 de la Competencia BCI IV. Extraído de [7].

El objetivo del desafío asociado a este dataset consiste en inferir la posición de cada uno de los dedos basándose exclusivamente en los datos de los electrodos, es decir, predecir los movimientos de la mano a partir de las señales cerebrales registradas. Este tipo de tarea es representativa de aplicaciones avanzadas de BCI en el control de prótesis o interfaces neuronales para personas con discapacidad motora.

3.1.1 Resultados de la BCI Competition IV - Dataset 4

En el desafío relacionado al Dataset 4 de la BCI Competition IV, el primer lugar fue obtenido por el equipo liderado por Nanying Liang, perteneciente al Cortex Team del Research Centre INRIA (Francia), en colaboración con Laurent Bougrain [5]. Su enfoque consistió en extraer características basadas en las amplitudes de la señal en distintas bandas de frecuencia: 1–60 Hz, 60–100 Hz y 100–200 Hz. Posteriormente, se realizó una selección de características basada en combinaciones de pares de electrodos (*pair-wise feature selection*) para optimizar la representatividad de las señales seleccionadas. Para la etapa de clasificación, emplearon un modelo de regresión lineal, implementado mediante la solución de Wiener y utilizando un retardo de 25 pasos (*25 tap delays*) para capturar la dinámica temporal de las señales cerebrales. Esta estrategia permitió modelar las relaciones temporales entre la actividad cerebral y el movimiento de los dedos de manera eficiente.

Cabe destacar que, en 2008, las redes neuronales profundas aún no eran una tecnología tan accesible ni madura como lo son en la actualidad. Por esta razón, la mayoría de los enfoques exitosos en aquella competencia, incluyendo el del equipo ganador que obtuvo un coeficiente de correlación de Pearson de 0.46 entre los valores reales y los predichos,

se basaban en métodos clásicos de procesamiento de señales y modelos lineales, los cuales ofrecían una combinación efectiva de desempeño y viabilidad computacional para el hardware disponible en ese entonces.

3.1.2 Resultados con técnicas del estado del arte

Desde ese entonces, el avance del aprendizaje profundo y de nuevas técnicas de representación ha permitido lograr una mejora sustancial en la decodificación de movimientos a partir de señales de ECoG. Buscando el dataset en la plataforma Papers with Code [10] es posible acceder a una tabla con papers que han utilizado el Dataset 4 de la competencia BCI IV [7], así como el coeficiente de Pearson que reportaron haber obtenido. En la Figura Aquí se mencionan brevemente los 3 mejores:

- En 2018, Xie et al. [16] realizó la extracción de características con una red neuronal convolucional (CNN) y el modelado temporal con una red neuronal recursiva del tipo Long Short-Term Memory (LSTM). Integrando ambas redes en una CNN-LSTM obtuvo un coeficiente de Pearson de 0.52.
- En 2022, Yao et al. [17] propuso otro enfoque sin el uso de redes neuronales profundas. En este caso, se combinaron características espectro-temporales clásicas (como potencia en bandas, LMP y parámetros de Hjorth) con representaciones geométricas basadas en matrices de covarianza proyectadas al espacio tangente de una variedad de Riemann. Para la etapa de regresión se utilizó el algoritmo Light Gradient-Boosting Machine (LightGBM) y se logró un coeficiente de Pearson de 0.53 sin una solución computacionalmente compleja y sin el uso de aprendizaje profundo para capturar las características ni las dinámicas temporales.
- En 2023, Lomtev et al. [6] implementó una arquitectura en un codificador-decodificador convolucional, es decir, una red neuronal que primero reduce progresivamente la dimensionalidad de la señal (codificación) y luego la reconstruye (decodificación), facilitando la extracción de patrones relevantes a diferentes escalas. Esta arquitectura incluyó conexiones extra que permiten que la información de capas iniciales se mantenga disponible durante la decodificación, mejorando la precisión espacial y la estabilidad del aprendizaje. Cabe mencionar que en este caso, antes de ser ingresadas al modelo, las señales ECoG fueron transformadas al dominio tiempo-frecuencia mediante la transformada de Wavelet de Morlet, centrándose principalmente en la banda de frecuencia que va entre los 40 y los 300 Hz, pues esta banda de frecuencias contiene información relacionada con actividad motora fina. La red no recibió la información temporal cruda. Este modelo alcanzó un coeficiente de Pearson de 0.67, el más alto reportado hasta la fecha para este dataset [10].

Brain Decoding on BCI Competition IV: ECoG to Finger Movements

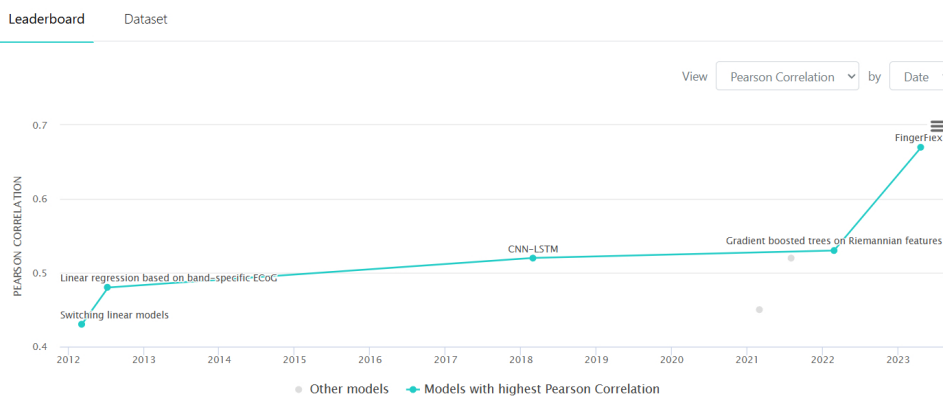


Figure 3: Gráfica que muestra los coeficientes de Pearson reportados para el dataset 4 de la competencia BCI IV [7] por distintos modelos a lo largo del tiempo. Extraído de [10].

3.2 Frameworks para TinyML

3.2.1 TensorFlow Lite

TensorFlow Lite (TFLite) [13] es una versión ligera de la librería TensorFlow, desarrollada por Google, diseñada específicamente para la ejecución de modelos de aprendizaje automático en dispositivos móviles, sistemas embebidos y microcontroladores. A diferencia de TensorFlow tradicional, que está optimizado para entrenar y ejecutar modelos en entornos de alto poder de cómputo, TensorFlow Lite está enfocado en ofrecer inferencias rápidas y eficientes en dispositivos con recursos limitados de procesamiento, memoria y energía.

En el contexto de TinyML, TensorFlow Lite for Microcontrollers (TFLM) es una variante aún más reducida de TFLite, pensada para microcontroladores que carecen de sistema operativo y disponen de apenas unos pocos kilobytes de memoria RAM. TFLM permite cargar modelos previamente entrenados y optimizados, normalmente mediante técnicas como cuantización, y ejecutarlos directamente en el dispositivo.

Cuando se utiliza TFLM en proyectos basados en STM32 mediante STM32CubeIDE, la integración debe realizarse de manera manual: es necesario importar la librería TFLM, configurar su ubicación en el linker y en el compilador, e incluir los encabezados correspondientes en el proyecto. Además, para que la inferencia funcione correctamente, se debe definir una “arena de memoria”, es decir, reservar explícitamente un bloque de memoria estática que será utilizado por TFLM para realizar los cálculos internos de la red neuronal. Determinar el tamaño adecuado de esta arena suele requerir un proceso de prueba y error, ajustando el tamaño hasta lograr un equilibrio entre estabilidad de ejecución y aprovechamiento de los recursos disponibles.

Una de las principales ventajas de TensorFlow Lite for Microcontrollers es su amplio soporte comunitario, su compatibilidad con diversas arquitecturas de hardware (ARM Cortex-M, RISC-V, entre otros) y la disponibilidad de herramientas de conversión que permiten transformar modelos de TensorFlow estándar en versiones compactas aptas para dispositivos de bajo recurso.

3.2.2 STM32Cube.AI

STM32Cube.AI [12] es una herramienta de conversión y optimización de redes neuronales desarrollada por STMicroelectronics, destinada a facilitar la implementación de modelos de aprendizaje automático en microcontroladores de la familia STM32. A diferencia de TensorFlow Lite for Microcontrollers, que actúa como una librería de inferencia generalizada, STM32Cube.AI se centra en traducir modelos ya entrenados a código C altamente optimizado, listo para ser ejecutado directamente en microcontroladores de la familia STM32.

Esta herramienta soporta modelos provenientes de diversos entornos de entrenamiento, como TensorFlow, Keras, PyTorch (exportado a formato ONNX) y Caffe, proporcionando una alta flexibilidad de integración. El flujo de trabajo habitual consiste en importar el modelo dentro de STM32CubeMX, configurar sus parámetros de despliegue, y generar automáticamente el código fuente necesario a partir del archivo `.ioc` del proyecto, integrándolo de forma transparente con STM32CubeIDE.

Una de las ventajas distintivas de STM32Cube.AI frente a TensorFlow Lite for Microcontrollers es su nivel de integración con el ecosistema de desarrollo STM32: el soporte para inferencia es proporcionado mediante paquetes de software que se agregan de forma asistida dentro del entorno de configuración, sin necesidad de realizar configuraciones manuales del linker o del compilador. Además, el espacio de memoria requerido para realizar los cálculos de la red neuronal no necesita ser definido manualmente mediante una arena de memoria; en su lugar, STM32Cube.AI genera una constante que indica automáticamente el tamaño de memoria necesario, basado en el análisis del modelo cargado. Esto permite una integración mucho más rápida y segura, evitando procesos de prueba y error asociados a la asignación de memoria. STM32Cube.AI también optimiza la ejecución del modelo para aprovechar características específicas del hardware STM32, como aceleradores matemáticos de punto flotante o unidades de procesamiento de enteros, y proporciona herramientas de validación que permiten medir la precisión y el rendimiento del modelo directamente en el dispositivo real.

4 Objetivos

4.1 Objetivo General

Implementar un sistema de clasificación de señales cerebrales basado en Edge Computing, implementado en la placa NÚCLEO-L476RG de STMicroelectronics, que asocie patrones de actividad cerebral a categorías de movimiento.

4.2 Objetivos Específicos

- Seleccionar y preprocesar un conjunto de datos de señales cerebrales adecuado para la clasificación de movimientos.
- Diseñar e implementar un modelo de Machine Learning en entorno computacional para la clasificación de las señales cerebrales.
- Adaptar y optimizar el modelo para su implementación en el microcontrolador STM32, considerando las restricciones de memoria y capacidad de procesamiento.
- Validar el funcionamiento del sistema integrado mediante:
 - La ejecución de inferencias en tiempo real en el microcontrolador
 - La medición de métricas de rendimiento (precisión y latencia)

5 Metodología

5.1 Hello World

5.1.1 Sine Model

En primer lugar, siguiendo la documentación oficial de TensorFlow Lite [13], se entrenó una red neuronal básica diseñada para aproximar la función seno. El esquema de esta red se muestra en la Figura 4, y corresponde a una red neuronal totalmente conectada (*fully connected*) de tres capas, que recibe un único número de punto flotante como entrada y entrega como salida una aproximación del valor de su seno. Una vez finalizado el entrenamiento, el modelo fue convertido desde su formato original de TensorFlow/Keras (.h5) a un modelo optimizado para TensorFlow Lite (.tflite), con el objetivo de permitir su implementación en microcontroladores.

Model: "sequential"

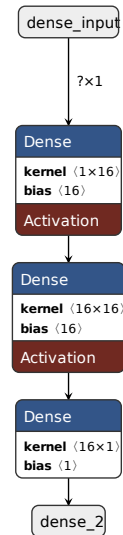
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 16)	32
dense_1 (Dense)	(None, 16)	272
dense_2 (Dense)	(None, 1)	17

Total params: 321 (1.25 KB)

Trainable params: 321 (1.25 KB)

Non-trainable params: 0 (0.00 B)

(a) Resumen del modelo en TensorFlow mostrando los parámetros de la red neuronal.



(b) Visualización de la arquitectura de la red utilizando Netron.app.

Figure 4: Modelo modelo_sine: Red neuronal totalmente conectada de tres capas.

Dado que inicialmente se tenía considerado usar TensorFlow Lite for Microcontrollers, también se generó de forma manual un archivo `.h` que contiene un arreglo estático que describe el modelo `.tflite` completo en formato binario. Finalmente se decidió utilizar STM32Cube.AI para facilitar la integración con STM32CubeIDE, por lo que la generación del archivo `.h` se vuelve innecesaria, ya que STM32Cube.AI recibe directamente el archivo `.tflite` o incluso el `.h5` sin optimizar. Si bien ya no será usado, vale la pena mencionar que el arreglo en el archivo `.h` no contiene únicamente los pesos de la red, sino que representa la estructura completa del modelo, incluyendo metadatos necesarios para su interpretación. Para utilizarlo, es imprescindible emplear funciones específicas de la biblioteca TensorFlow Lite Micro, ya que el modelo requiere ser interpretado en tiempo de ejecución para su correcto funcionamiento en el dispositivo embebido, por eso se decidió descartarlo.

Ya que se estaba siguiendo la documentación de TensorFlow Lite [13], originalmente se entrenó la red neuronal con un dataset que consiste en los pares $(x, y) = (x, \sin(x) + 0.1 \cdot \mathcal{N}(0, 1))$, pero se vio que este ruido simplemente estaba añadido con motivos didácticos, por lo que se decidió eliminarlo para poder comparar más fácilmente el desempeño de la red implementada en el microcontrolador con respecto a la red implementada en el computador al simplemente calcular el error absoluto medio entre la salida de la red y la función $\sin(x)$.

5.1.2 Fashion NN y Fashion CNN

Dado que las señales neuronales contienen información distribuida a través de múltiples canales, y no es suficiente con analizarlos de forma individual, se consideró utilizar redes capaces de detectar relaciones espaciales y temporales de sin tener que codificar este valor en la entrada (el mejor ejemplo de esto es el uso de las CNN-LSTM por Xie et al.[16]). Por esto, se entrenaron también otras 2 redes neuronales como parte de la fase inicial del *Hello World* del proyecto, estas fueron Fashion NN, una red neuronal *fully connected* y Fashion CNN, una red neuronal convolucional. Ambas fueron entrenadas para clasificar el dataset Fashion MNIST, un conjunto de datos de imágenes en escala de grises de 28x28 píxeles, representando distintos tipos de prendas de vestir [15]. Finalmente se decidió no proseguir con estas redes neuronales, utilizándose solamente el modelo del seno para el *Hello World* y la red *fully connected* con entradas preprocesadas como se describirá en la Sección 5.2 para el problema central.

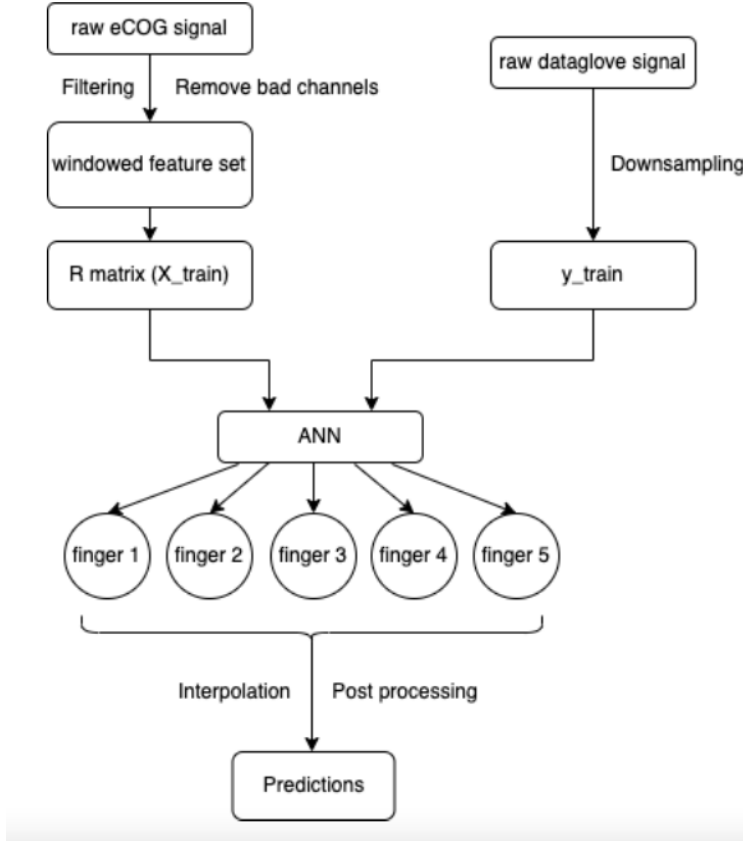
5.2 Finger Flexion Detection Using ECoG Signal

En paralelo con el desarrollo y entrenamiento de las redes “Hello World”, se exploraron implementaciones de modelos basados en redes neuronales que utilizaran el dataset seleccionado [7]. Se priorizó la búsqueda de modelos que emplearan arquitecturas de redes neuronales, con el fin de aprovechar las herramientas modernas de TinyML, diseñadas específicamente para optimizar y reducir redes, en lugar de modelos tradicionales de Machine Learning. En este contexto, se identificó el proyecto de raimasen1729 [11], cuyo flujo de procesamiento se presenta en la Figura 5a. Este esquema contempla un extenso preprocesamiento de los datos de entrada, que consiste en los siguientes pasos:

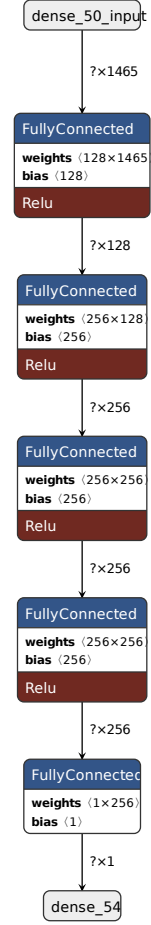
1. Eliminación de canales defectuosos, dependiendo del sujeto.
2. Aplicación de un filtro FIR pasabanda para mantener solo la información en el rango 4–200 Hz, eliminando componentes de baja frecuencia y altas frecuencias contaminadas por ruido.
3. Aplicación de filtros notch centrados en 60 Hz, 120 Hz y 180 Hz para remover la interferencia de la red eléctrica (y sus armónicos), acorde a la frecuencia de operación en Estados Unidos, donde fue adquirido el dataset.
4. División de los datos en ventanas deslizantes y extracción, para cada ventana, de una serie de características:
 - Voltaje promedio.
 - Longitud de línea (variación acumulada de la señal).
 - Energía total de la señal.
 - Energía promedio en los rangos 5–15 Hz, 20–25 Hz, 75–115 Hz, 125–160 Hz y 160–175 Hz.
5. Construcción de una matriz de regresores (*R matrix*), concatenando las características de las últimas tres ventanas, para otorgar información temporal al modelo.

En cuanto a los datos de etiquetas, se hizo un downsampling para facilitar el procesamiento de los datos de flexión. Esto tiene sentido porque el guante utilizado para estas mediciones opera a 25Hz, mientras que los datos del dataset se encuentran a 1kHz.

Finalmente, las matrices de regresores y las etiquetas de flexión fueron utilizadas para entrenar cinco redes neuronales, cada una enfocada en predecir la posición de un dedo específico. Posteriormente, las predicciones fueron concatenadas e interpoladas mediante splines cúbicos para presentar los resultados en la misma frecuencia que los datos de prueba.



(a) Diagrama de flujo del algoritmo de raimasen1729. Extraído de [11].



(b) Visualización de la arquitectura de la red para un dedo del sujeto 1, usando Netron.app.

Figure 5: Diagramas del flujo del algoritmo y de la arquitectura de red utilizada para la predicción de la flexión de los dedos.

5.2.1 Modificaciones iniciales al modelo

Antes de transformar las redes neuronales de este algoritmo al formato `.tflite`, fue necesario realizar una serie de modificaciones:

- Aunque el diagrama en [11] sugiere la existencia de cinco redes neuronales independientes, una para cada dedo, en la práctica el entrenamiento reutilizaba una misma instancia de red de manera secuencial para los cinco dedos. En lugar de entrenar cinco redes independientes, cada una optimizada para un dedo, se utilizaba una única red: primero se entrenaba con los datos del primer dedo y se guardaba el modelo, luego se continuaba el entrenamiento con los datos del segundo dedo, y así sucesivamente, sin reinicializar los pesos en ningún momento. Este enfoque implica que la red correspondiente al primer dedo fue entrenada únicamente con datos de ese dedo, mientras que la red final para el quinto dedo acumuló información proveniente de todos los dedos anteriores. Como consecuencia, existe un riesgo de falta de generalización en los modelos iniciales y de

contaminación en los modelos finales, dado que las mismas señales cerebrales podían asociarse a distintos niveles de flexión en diferentes etapas del entrenamiento, sin que la red tuviera conocimiento explícito de a qué dedo correspondía cada flexión.

- Adicionalmente, se corrigió el procedimiento de partición entre datos de entrenamiento y validación. Debido a la naturaleza temporal de los datos, no era adecuado seleccionar un 10% aleatorio: las dependencias temporales en la matriz de regresores podían provocar fuga de información. En su lugar, se utilizó el último 10% de los datos para validación, garantizando una separación temporal adecuada entre entrenamiento y validación.
- Finalmente, se realizaron otras modificaciones menores antes de entrenar la versión del modelo que fue integrada en STM32Cube.AI (cuya arquitectura se muestra en la Figura 5b):
 - Incorporación de capas de **Dropout** para mitigar el sobreajuste (*overfitting*).
 - Aplicación de **regularización L2** para controlar la magnitud de los pesos.
 - Realización de un **escalamiento Min-Max** de las etiquetas de flexión al rango [0, 1], para evitar sesgos derivados de la distribución original de los datos.
 - Modificación del método de reducción de ventanas de datos, utilizando el **promedio** en lugar del **máximo**, con el objetivo de hacer las características más robustas al ruido.
 - Introducción de un **desfase temporal** de una muestra entre las señales ECoG y las mediciones de flexión, siguiendo las recomendaciones de la descripción del dataset [7], para reflejar el retardo biológico entre la actividad cerebral y el movimiento físico.

Con estas modificaciones se entrenó un modelo que alcanzaba un coeficiente de Pearson de 0.29 para el sujeto 1, pero que no era capaz de entrar en el STM32.

5.2.2 Refinación del modelo

Durante el proceso de exploración de distintas opciones para reducir el tamaño del modelo, una de las primeras opciones que fueron consideradas tras la cuantización, fue cambiar del sujeto 1 al sujeto 2 del dataset, pues el sujeto 1 tenía 62 electrodos implantados en su córtex, mientras que el sujeto 2 contaba con 48. Esta diferencia en la cantidad de los canales de entrada se traduce en una diferencia entre una matriz de regresores con 1465 columnas y una con 1153 columnas, lo cual reduciría el tamaño de la capa inicial.

Tras cambiar de sujeto se notó que las redes neuronales entrenadas con los datos del sujeto 2 lograban un desempeño inferior a aquellas entrenadas con los datos del sujeto 1, por lo que se decidió volver a modificar la red neuronal para mejorar el coeficiente de Pearson. Las modificaciones implementadas fueron las siguientes:

- Se probaron varios valores de **Dropout**, para disminuir el sobreajuste, tal como se había hecho originalmente.
- El modelo original ya incorporaba **Early Stopping**, una técnica que se asegura de detener el entrenamiento cuando se detecta que está empezando a sobreajustarse, pero este entrenamiento se detenía al rededor de la época 80 de las 500 épocas que se habían definido como máximas, por lo que se agregó **Reducing Learning Rate**, una técnica que hace que entre más épocas hayan transcurrido en el entrenamiento, los pasos que da el modelo hacia la solución óptima sean cada vez más cortos, lo que hace que cada vez sea “más cuidadoso” y hace más posible aprender más antes de llegar al sobreajuste.
- Finalmente, la técnica que mejor ayudó a mejorar el desempeño de la red neuronal, además de disminuir drásticamente su peso, fue disminuir la cantidad de neuronas. La versión original de la red, como se puede ver en la Figura 5b, contaba con 4 capas internas, una de 128 neuronas y las otras 3 con 256, lo que hacía que esta red fuese muy propensa al sobreajuste por ser innecesariamente compleja para la cantidad de datos disponibles. Tras probar diversas combinaciones e ir reduciendo las capas y la cantidad de neuronas, se llegó a la arquitectura mostrada en la Figura 6, una red con tan solo 2 capas internas, una de 128 neuronas y la otra de 64.

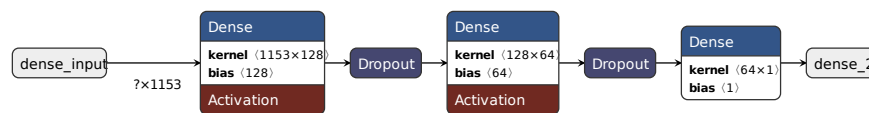


Figure 6: Visualización de la nueva arquitectuara de la red para un dedo del sujeto 2, usando Netron.app.

5.3 Preparación de modelos en STM32Cube.AI

Una vez entrenados los distintos modelos a probar, estos eran convertidos al formato TensorFlow Lite (.tflite), en versión de precisión completa y cuantizada a 8 bits y fueron integrados en un proyecto STM32 utilizando la herramienta STM32Cube.AI. La carga de los modelos en STM32Cube.AI permitió generar automáticamente reportes de complejidad, utilización de memoria y tiempos estimados de inferencia para cada uno de ellos. Esta información fue utilizada posteriormente para evaluar la viabilidad de implementar cada red en un entorno de recursos restringidos, y se presenta en detalle en la sección de Resultados.

Un paso clave en esta preparación, en especial en el caso de los modelos cuantizados, es la selección de un conjunto de datos representativo. Durante el proceso de cuantización, se utiliza una pequeña muestra de los posibles valores de entrada de la red neuronal. Esta muestra permite al cuantizador estimar adecuadamente tanto los rangos que debe ser capaz de manejar la red en sus entradas como la magnitud esperada de sus salidas. Suministrar este dataset representativo es fundamental para asegurar que el modelo cuantizado sea capaz de generar salidas en un rango coherente y funcional.

5.4 Implementación de la red neuronal

A continuación se explicaran algunas secciones del código necesario para implementar cualquiera de estos modelos en el proyecto para STM32. Esta lista muestra los nombres y valores utilizados para `sine_model`, el modelo *Hello World*, pero para cualquier otro modelo, se deben seguir las mismas direcciones, solo que utilizando el nombre que se le haya asignado a la red tras entregarle el modelo a STM32Cube.AI. Todas las líneas mencionadas a continuación se encuentran en el archivo `main.c`:

- En la sección `USER CODE Includes`, incluir:
 - `"ai_datatypes_defines.h"`
 - `"ai_platform.h"`
 - El modelo de la red neuronal con el nombre que se haya definido en el STM32CubeMX, por ejemplo, `"sine_model.h"`
 - Y los metadatos asociados con ese mismo nombre, por ejemplo, `"sine_model_data.h"`
- Dentro de `USER CODE 1`, en `main()` hay que definir algunos espacios de memoria para poder utilizar la red neuronal más adelante, tales como:
 - Un espacio de memoria para guardar calculos intermedios. El tamaño de este espacio está determinado por variables definidas por Cube.AI, no por el usuario.


```
* AI_ALIGNED(4) ai_u8 activations[AI_SINE_MODEL_DATA_ACTIVATIONS_SIZE];
```
 - Espacios de memoria para los tensores de entradas y salidas:


```
* AI_ALIGNED(4) ai_float in_data[AI_SINE_MODEL_IN_1_SIZE];
* AI_ALIGNED(4) ai_float out_data[AI_SINE_MODEL_OUT_1_SIZE];
```
 - Es necesario definir el manejador (`handle`) y los parámetros para poder utilizarlos luego al inicializar la red:


```
* ai_handle sine_model = AI_HANDLE_NULL;
* ai_network_params ai_params = {
    .params = AI_SINE_MODEL_DATA_WEIGHTS(ai_sine_model_data_weights.get()),
    .activations = AI_SINE_MODEL_DATA_ACTIVATIONS(activations),
};
```
 - Hay que definir punteros hacia entradas y salidas:


```
ai_buffer ai_input[AI_SINE_MODEL_IN_NUM];
ai_buffer ai_output[AI_SINE_MODEL_OUT_NUM];
```
 - Y finalmente definir las estructuras de los buffers de entrada y salida como tales:


```
* ai_input[0].data = AI_HANDLE_PTR(in_data);
* ai_input[0].shape.size = 4;
* ai_input[0].shape.data = {1,1,1,1};
* ai_input[0].format = AI_BUFFER_FORMAT_FLOAT;
* Y lo mismo para ai_output[0].
```

- Cabe mencionar que esto es para el caso específico para redes con una entrada escalar y una salida escalar. La razón por la que se considera esta entrada como un tensor de 4 dimensiones de tamaño 1 es porque estas dimensiones representan el número de muestras, la altura, el ancho y los canales (N,H,W,C). Así, para 1 escalar, la forma de los datos de entrada es $\{1, 1, 1, 1\}$, mientras que para un batch de 4 imágenes en blanco y negro de 28x28 bits sería de $\{4, 28, 28, 1\}$. En el caso de la red de la Figura 6, se tiene considera que la entrada es 1 muestra de datos de 1153x1, de 1 canal. $\{1, 1153, 1, 1\}$
- Dentro de USER CODE 2 en main hay que crear e inicializar la red neuronal usando:
 - `ai_sine_model_create(&sine_model, AI_SINE_MODEL_DATA_CONFIG);`
 - `ai_sine_model_init(sine_model, &ai_params)`
- Finalmente, para utilizar la red, dentro del mismo USER CODE 2 o en USER CODE WHILE, se realiza la inferencia con:
 - `ai_sine_model_run(sine_model, &ai_input[0], &ai_output[0]);`

Recordar que el nombre de casi todas las funciones y parámetros aquí mencionados depende del nombre que se le haya dado a la red neuronal en la sección de STM32Cube.AI del STM32CubeMX.

5.5 Evaluación de los modelos

5.5.1 Evaluación del modelo Hello World

Para la evaluación del modelo entrenado para calcular el seno de la entrada, se decidió no solo hacer pruebas de qué tanto se degrada el desempeño de la red neuronal al disminuir la precisión, sino también al pasar esta desde el computador al microcontrolador. Utilizando el modelo .h5, sin cuantizar, en el PC y el modelo .tflite, sin cuantizar, en el STM32, se corroboró que las salidas de ambas redes neuronales eran exactamente iguales.

w

Tras hacer esta prueba, se escribió un programa en Python que envía 10.000 valores por UART al STM32 con el modelo cuantizado y se compararon sus salidas tanto con el modelo completo en el computador, como con la sinusoidal real y se calculó el error absoluto medio entre ambas redes neuronales y la sinusoidal. También se comprobó el tiempo de inferencia en el STM32.

5.5.2 Evaluación del modelo Finger Flexion Detection

El desafío original para el que fue construido el dataset seleccionado [7] consistía en predecir la posición de los dedos para los últimos 3 minutos y 20 segundos de las señales grabadas y la métrica de éxito corresponde a la media aritmética de los coeficientes de correlación de Pearson entre las posiciones predichas para los dedos y las posiciones reales de estos (sin contar el dedo anular), por lo que el desempeño tanto de la red neuronal completa en el PC como de la red neuronal cuantizada e implementada en el microcontrolador, será medido en primera instancia según este coeficiente. Además, como una prueba extra, se decide hacer pruebas para medir el error absoluto medio entre la red neuronal original y la cuantizada.

La forma en que se hacen estas pruebas es enviando via UART las filas de la matriz de regresores (R) para las señales cerebrales de estos últimos 3 minutos y 20 segundos y se reciben via UART las posiciones de los 5 dedos. Originalmente se tenía considerado hacer pruebas de latencia también, pero el código actual contiene un gran cuello de botella en la fase de enviar los datos, pues además de ser 3996 muestras de 1153 flotantes cada uno, el código no se encuentra optimizado con DMA ni con interrupciones para obtener un throughput aceptable. Actualmente se envían 1153 flotantes y se reciben 5 flotantes, luego se envían otros 1153 flotantes y se reciben 5, y así hasta completar las 3996 muestras que corresponden a los 3 minutos y 20 segundos. En versiones posteriores esto se podría mejorar para que las redes neuronales estén procesando una entrada mientras la UART recibe la siguiente.

6 Resultados

6.1 Hello World

6.1.1 Sine Model

Tras el entrenamiento, se realizó la inferencia en el mismo computador utilizando el modelo completo, sin aplicar optimizaciones ni reducciones, obteniéndose el gráfico mostrado en la Figura 7. Para esta prueba se utilizaron 10.000 puntos equiespaciados en el rango $[-1, 2\pi + 1]$, cabe mencionar que el dataset consistía de 1.000 valores distribuidos aleatoriamente de manera uniforme entre 0 y 2π , se añadieron puntos a la prueba para ver qué tan bien generaliza el modelo y se calculó un error absoluto medio de 0.0574 para el rango $[-1, 2\pi + 1]$, mientras que al evaluar la red neuronal con 10.000 puntos en el intervalo $[0, 2\pi]$, se obtiene un error absoluto medio de 0.02560.

Se puede notar fácilmente que la red neuronal generalizó bastante bien para puntos nuevos en el rango original, pero no muy bien para los puntos por debajo del 0 ni por encima de los 2π . Podría decirse que interpola relativamente bien, pero no extrapola muy bien.

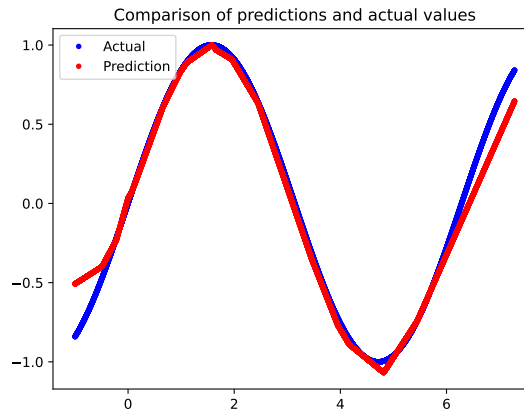


Figure 7: Resultados de la inferencia del modelo `model_sine` en comparación con los valores reales del seno de x .

Tras las pruebas en el computador, se integró el modelo en STM32Cube.AI, obteniéndose un uso de 9.89 KiB de memoria Flash (aproximadamente un 0.97% del 1MiB de Flash total) y 1.89 KiB de memoria RAM (aproximadamente un 1.48% de los 128KiB de RAM totales), además de una complejidad de 353 operaciones de multiplicación y acumulación (MACC), por lo que se nota que se trata de un modelo extremadamente liviano y que no necesita ser cuantizado para su implementación en el microcontrolador utilizado.

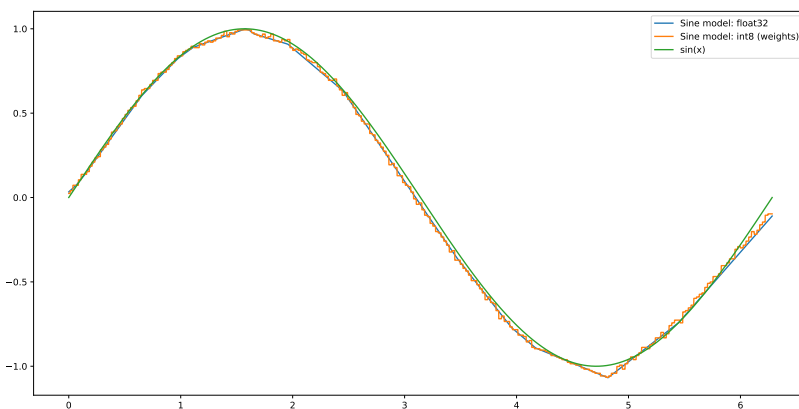
Tras hacer cuantización de los pesos se registra un aumento tanto en el uso de memoria Flash, a 19.52KiB de memoria Flash (1.91% del total), como en el uso de memoria RAM, a 2.05KiB (1.6% del total) y una disminución en la complejidad, a 325 MACC. La razón de este aumento se vuelve aparente al revisar en detalle el informe con el uso de memoria del modelo. Como se puede notar en la Figura 8, esto se debe a que, si bien los pesos utilizan un cuarto del espacio en memoria Flash en la versión cuantizada, el espacio utilizado por las librerías necesarias para utilizar el modelo es más grande con la versión cuantizada tanto en Flash como en RAM.

Por último, en la Figura 9 se muestran las salidas de la red con precisión completa, junto con la cuantizada, ambas implementadas en el STM32, y la función $\sin(x)$ para el rango $[0, 2\pi]$. La red neuronal con la precisión completa obtuvo un error absoluto medio de aproximadamente 0.02560 y la de pesos de 8 bits uno de aproximadamente 0.02483, sorpresivamente, esto es ligeramente mejor que la de precisión completa, pero en algunas partes de la figura se puede notar que el mismo ruido de la cuantización puede hacer que la salida cuantizada se encuentre más cerca de la señal real que la salida original.

Se midió también el tiempo de inferencia y se determinó que son $90\mu s$ al usar un reloj de $80MHz$, es decir, la inferencia toma 7200 ciclos de reloj.

(a) Sección final del análisis de la red `sine_model` con precisión de 32 bits.

(b) Sección final del análisis de la red `sine_model` con precisión de 8 bits.



```

Summary - "stm3214" target
-----
FLASH (ro)    %*    RAM (rw)    %
-----
RT total      10,004  0.8%    2,804  36.2%
-----
TOTAL         1,235,740          7,736
-----
* rt/total
Creating txt report file C:\Users\david\.stm32cube\stm32cube\network
elapsed time (analyze): 77.401s
Model file:    model_s2f1.tflite
Total Flash:   1235736 B (1.18 MiB)
Weights:       1225732 B (1.17 MiB)
Library:       10004 B (9.77 KiB)
Total Ram:     7736 B (7.55 KiB)
Activations:   4932 B (4.82 KiB)
Library:       2804 B (2.74 KiB)
Input:         4420 B (4.32 KiB included in Activations)
Output:        4 B (included in Activations)
Done
Analyze complete on AI model
Required Ram or Flash size for network_1 is bigger than available Ram or Flash

```

(a) Versión inicial del modelo, 32 bits, sujeto 2.

```

-----
FLASH (ro)    %*    RAM (rw)    %
-----
RT total      16,057  7.4%    2,212  27.1%
-----
TOTAL         215,681          8,168
-----
* rt/total
Creating txt report file C:\Users\david\.stm32cube\stm32cube\model_
elapsed time (analyze): 16.290s
Model file:    model_s1f1_int8.tflite
Total Flash:   215677 B (210.62 KiB)
Weights:       199620 B (194.94 KiB)
Library:       16057 B (15.68 KiB)
Total Ram:     8168 B (7.98 KiB)
Activations:   5956 B (5.82 KiB)
Library:       2212 B (2.16 KiB)
Input:         5956 B (5.82 KiB included in Activations)
Output:        4 B (included in Activations)
Done
Analyze complete on AI model

```

(c) Versión final del modelo, 8 bits, sujeto 1.

```

-----
FLASH (ro)    %*    RAM (rw)    %
-----
RT total      8,772  1.1%    1,812  21.9%
-----
TOTAL         804,940          8,280
-----
* rt/total
Creating txt report file C:\Users\david\.stm32cube\stm32cube\model_s1f1
elapsed time (analyze): 68.454s
Model file:    model_s1f1.tflite
Total Flash:   804936 B (786.07 KiB)
Weights:       796164 B (777.50 KiB)
Library:       8772 B (8.57 KiB)
Total Ram:     8280 B (8.09 KiB)
Activations:   6468 B (6.32 KiB)
Library:       1812 B (1.77 KiB)
Input:         5956 B (5.82 KiB included in Activations)
Output:        4 B (included in Activations)
Done
Analyze complete on AI model

```

(b) Versión final del modelo, 32 bits, sujeto 1.

```

Summary - "stm3214" target
-----
FLASH (ro)    %*    RAM (rw)    %
-----
RT total      15,651  9.1%    2,040  30.7%
-----
TOTAL         172,267          6,652
-----
* rt/total
Creating txt report file C:\Users\david\.stm32cube\stm32cube\model_
elapsed time (analyze): 65.387s
Model file:    model_s2f1_int8.tflite
Total Flash:   172263 B (168.23 KiB)
Weights:       156612 B (152.94 KiB)
Library:       15651 B (15.28 KiB)
Total Ram:     6652 B (6.50 KiB)
Activations:   4612 B (4.50 KiB)
Library:       2040 B (1.99 KiB)
Input:         4612 B (4.50 KiB included in Activations)
Output:        4 B (included in Activations)
Done
Analyze complete on AI model

```

(d) Versión final del modelo con precisión 8 bits y sujeto 2. Esta es la versión implementada.

Figure 10: Detalle del análisis del uso de memoria de distintas versiones de las redes para detectar la flexión de los dedos. Estos informes fueron generados automáticamente por STM32Cube.AI.

Model manager			
Name	Used Ram	Used Flash	Complexity
model_s2f1	4.50 KiB	171.75 KiB	158341 MACC
model_s2f2	4.50 KiB	171.75 KiB	158341 MACC
model_s2f3	4.50 KiB	171.75 KiB	158341 MACC
model_s2f4	4.50 KiB	171.75 KiB	158341 MACC
model_s2f5	4.50 KiB	171.75 KiB	158341 MACC
Library	1.99 KiB	.00 B	
Total (5)	24.51 KiB	858.73 KiB	791705 MACC

Figure 11: Memoria total necesaria para implementar las redes independientes para los cinco dedos en su versión final implementada, incluyendo la complejidad en términos de operaciones MACC.

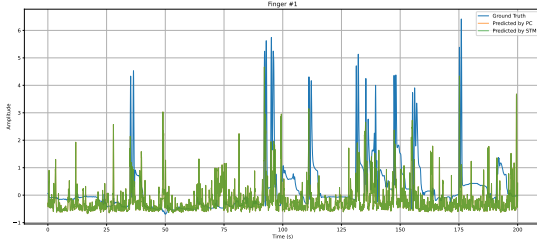
7 Discusión

7.1 Sine Model

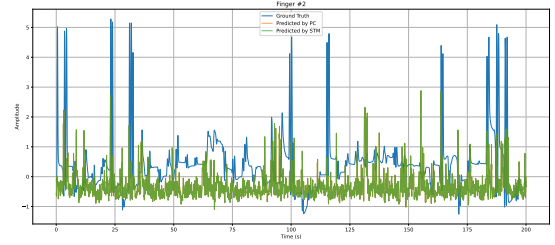
La conclusión más relevante a extraer de los resultados es que cuando los modelos son lo suficientemente pequeños, no vale la pena cuantizar, pues si bien el espacio ocupado por los pesos se reduce a un cuarto del original, el overhead asociado a la librería para hacer que el modelo pueda ser utilizado es mayor con los modelos de pesos de 8 bits que con los de pesos de 32 bits. Una de las razones por las que ocurre esto es que, al tener una red que recibe entradas de 32 bits, pero usa pesos de 8, es necesario que esta haga internamente la conversión de 32 a 8 para las entradas y de vuelta de 8 a 32 en las salidas. Si en vez de cuantizar solo los pesos se cuantizan también las funciones de activación y las entradas y salidas, el tamaño de la librería también aumenta con respecto a la versión original, pero no tanto

Coefficiente de Pearson	GT vs PC	GT vs STM	PC vs STM
Dedo 1	0.356	0.356	0.999
Dedo 2	0.194	0.203	0.868
Dedo 3	0.095	0.103	0.776
Dedo 4	0.283	0.289	0.983
Dedo 5	0.148	0.148	0.999
Promedio (sin contar el anular)	0.1982	0.2027	0.9106

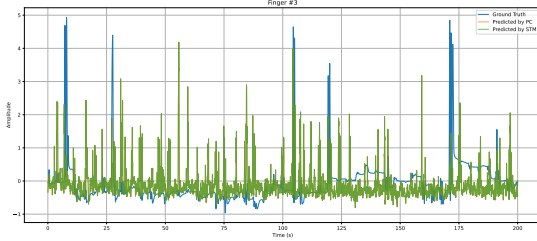
Table 1: Coeficientes de correlación de Pearson entre las etiquetas reales (GT), las predicciones hechas en el computador (PC) y las predicciones hechas con el modelo en el microcontrolador (STM) para cada dedo del sujeto 2.



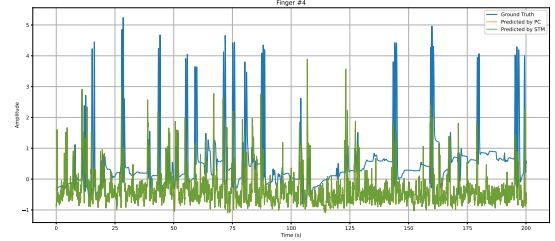
(a) Posiciones para el pulgar.



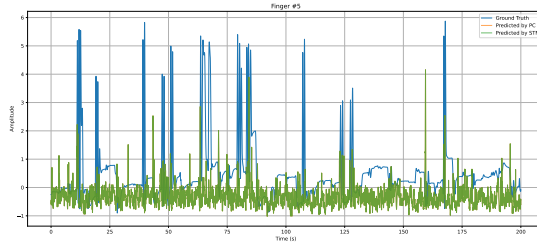
(b) Posiciones para el índice.



(c) Posiciones para el dedo del medio.



(d) Posiciones para el anular.



(e) Posiciones para el meñique.

Figure 12: Posición real y predicha para cada dedo del sujeto 2.

como en la versión en que solo los pesos son cuantizados.

Otra cosa interesante fue que el error absoluto medio de la versión cuantizada resultó ser ligeramente mayor que el de la versión original. Si esta diferencia hubiese sido mayor, se entiende que ha habido un error, pues eso no debería pasar, pero como la diferencia fue mínima, esto no es un problema. Dos posibles explicaciones son, por un lado, que la cuantización actúa como una especie de regularización de los pesos, lo que puede reducir levemente el sobreajuste, y la otra opción tiene que ver con el ruido que se introduce al cuantizar el modelo y la distribución de los puntos del dataset representativo que fue utilizado para la cuantización.

7.2 Finger Flexion

El que una red neuronal sea demasiado grande para el hardware no necesariamente significa que sea necesario cambiar de hardware. En este proyecto, la versión original del modelo no cabía en el microcontrolador STM32 cuando se usaba con precisión completa de 32 bits. Si bien al cuantizar a 8 bits fue posible hacer funcionar una o incluso dos redes, implementar las redes asociadas a los 5 dedos del sujeto 1 seguía superando las capacidades de memoria disponibles.

Este problema motivó una exploración más amplia de técnicas de optimización como la cuantización extrema (a 4 o incluso 1 bit por peso) y el knowledge distillation, con el fin de reducir el tamaño del modelo manteniendo su rendimiento. Sin embargo, más allá de aplicar métodos avanzados de TinyML, lo que solucionó el problema fue volver a examinar cuidadosamente el modelo original. Se identificó que una parte importante del sobreajuste provenía del tamaño excesivo del modelo en relación con el problema y el conjunto de datos.

Finalmente, en vez de técnicas de TinyML, la solución adoptada fue cambiar del sujeto 1 al sujeto 2 para reducir la dimensionalidad de las entrada de la red, así como también ajustar el tamaño de la red neuronal para mejorar su desempeño. Esto no solo permitió evitar el sobreajuste, sino que también se llegó a una arquitectura que mucho más liviana con la que sí fue posible implementar las cinco redes cuantizadas a 8 bits para cumplir el objetivo propuesto, ocupando apenas un 19% de la RAM disponible y un 84% del total de memoria flash del STM32.

Este caso nos recuerda que, en muchas ocasiones, en lugar de recurrir de inmediato a técnicas complejas para resolver un problema, es más efectivo volver a lo fundamental y revisar la base del diseño. Corregir o simplificar los aspectos esenciales puede ser la clave para alcanzar una solución satisfactoria.

8 Trabajo Futuro

Si bien tanto el objetivo general como los objetivos específicos fueron logrados, todavía quedan varias cosas que podrían incorporarse para hacer un sistema más avanzado y más práctico, considerando que la idea original tenía relación con evaluar la viabilidad de implementar Edge Computing para el control de prótesis biónicas.

A continuación se muestran posibles soluciones tanto para mejorar el rendimiento temporal como para hacer que el sistema sea más interactivo, pero vale la pena mencionar que la mayoría de estas soluciones probablemente requerirían un microcontrolador más avanzado y sobre todo, con mayor memoria Flash.

8.1 Soluciones para disminuir latencia y aumentar throughput

Sin hacer el procesamiento de las señales cerebrales en tiempo real, el modelo actual demora al rededor de 2 horas en predecir la posición de 5 dedos durante 3 minutos y 20 segundos. Esto se debe al gran cuello de botella que hay en el envío y recepción de datos, pues en cada paso el microcontrolador debe recibir 1153 datos de 4 bytes y actualmente espera a terminar de recibirlos antes de entregar las salidas de los 5 dedos. Algunas formas de mejorar el desempeño en esta área son:

- **Incorporar el preprocesamiento**, o al menos parte de él, en el microcontrolador. Si el microcontrolador fuese capaz de hacer todo el procesamiento, solo tendría que recibir 48 datos de 4 bytes por vez, por lo que incluso si el proceso siguiera siendo totalmente secuencial, el cuello de botella sería menor.
- **Incorporar DMA** para recibir los datos. Si se modifica el código para estar recibiendo los datos del siguiente ciclo mientras la red neuronal está procesando el ciclo actual sería posible disminuir el throughput incluso con la versión que envía 1153 datos por ciclo, pero si esto se combina con hacer parte del procesamiento de los datos en el mismo microcontrolador, esto podría mejorar mucho tanto la latencia como el throughput del sistema.
- **Implementar una red neuronal más moderna**. Con redes neuronales más especializadas que una simple red *fully connected*, es posible hacer que la misma red neuronal haga parte del preprocesamiento, disminuyendo tanto la cantidad de entradas a recibir, como el preprocesamiento extra que sería necesario hacer en el microcontrolador. Implementar un codificador-decodificador convolucional como el de Lomtev et al. [6] mejoraría en gran manera el desempeño, pero incluso implementar una CNN-LSTM como la de Xie et al. [16], mejoraría grandemente el rendimiento temporal del sistema, pues es la misma red neuronal la que se encargaría de extraer las características e identificar las relaciones temporales, sin tener que enviar un vector de regresores con más de 1000 columnas por cada paso.

8.2 Soluciones para mejorar interactividad

Por último, sería interesante también incorporar alguna forma de observar la predicción de las posiciones de los dedos de una forma más interactiva y visual que simplemente calcular el coeficiente de Pearson y observar los gráficos de las salidas procesadas.

Dos formas en que se podría abordar esto serían:

1. Procesar las salidas en el computador después de recibirlas todas e implementar una animación para mostrar el movimiento real de la mano y el movimiento predicho, algo como lo que se muestra en el repositorio de FingerFlex [6].
2. Realizar el procesamiento en el mismo microcontrolador e incorporar alguna forma de visualizar esto, que podría ser algo tan simple como luces que aumentan o disminuyen su luminosidad según la posición predicha, o algo más mucho más complejo como usar servomotores para mostrar la posición aproximada de los dedos. Esta segunda opción requeriría disminuir la latencia del sistema para funcionar de manera satisfactoria.

References

- [1] Berlin Brain-Computer Interface. *About the BBCI Project*. Visitado el: 28-05-2025. 2024. URL: <https://bbci.de/about>.
- [2] Edge Computing vs. Cloud Computing: What's the Difference? Accessed: 2025-04-17. Fauna. Sept. 26, 2021. URL: <https://fauna.com/blog/edge-computing-vs-cloud-computing-whats-the-difference>.
- [3] Jianping Gou et al. "Knowledge Distillation: A Survey". In: *International Journal of Computer Vision* 129 (2021), pp. 1789–1819. DOI: [10.1007/s11263-021-01453-z](https://doi.org/10.1007/s11263-021-01453-z). URL: <https://doi.org/10.1007/s11263-021-01453-z>.
- [4] Itay Hubara et al. "Quantized Neural Networks: Training Neural Networks with Low Precision Weights and Activations". In: *Journal of Machine Learning Research* 18.187 (2018), pp. 1–30. URL: <http://jmlr.org/papers/v18/16-456.html>.
- [5] Nanying Liang and Laurent Bougrain. "Decoding finger flexion from band-specific ECoG signals in humans". In: *Frontiers in neuroscience* 6 (2012), p. 91.
- [6] Vladislav Lomtev, Alexander Kovalev, and Alexey Timchenko. "FingerFlex: Inferring finger trajectories from ECoG signals". In: *arXiv preprint arXiv:2211.01960* (2022).
- [7] Kai J Miller and Gerwin Schalk. "Prediction of finger flexion: 4th brain-computer interface data competition". In: *BCI Competition IV* 1 (2008), pp. 1–2.
- [8] Pavlo Molchanov et al. "Importance estimation for neural network pruning". In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 11264–11272.
- [9] Markus Nagel et al. *A White Paper on Neural Network Quantization*. 2021. arXiv: [2106.08295](https://arxiv.org/abs/2106.08295) [cs.LG]. URL: <https://arxiv.org/abs/2106.08295>.
- [10] Papers with Code. *State-of-the-art Results for Brain Decoding on BCI Competition IV (ECoG to Finger Flexion)*. <https://paperswithcode.com/sota/brain-decoding-on-bci-competition-iv-ecog-to>. Accessed: 2025-05-29. 2025.
- [11] raimasen1729. *Finger Flexion Detection Using ECoG Signal*. <https://github.com/raimasen1729/Finger-Flexion-Detection-Using-ECoG-Signal>. Accessed: 2025-03-25. 2022.
- [12] STMicroelectronics. *STM32Cube.AI User Manual*. Accessed: 2025-04-15. 2025. URL: <https://www.st.com/en/embedded-software/x-cube-ai.html>.
- [13] TensorFlow Authors. *TensorFlow Lite Documentation*. Accessed: 2025-03-31. 2025. URL: <https://www.tensorflow.org/lite/guide?hl=es-419>.
- [14] Sunil Vadera and Salem Ameen. "Methods for Pruning Deep Neural Networks". In: *IEEE Access* 10 (2022), pp. 63280–63300. DOI: [10.1109/ACCESS.2022.3182659](https://doi.org/10.1109/ACCESS.2022.3182659).
- [15] Han Xiao, Kashif Rasul, and Roland Vollgraf. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. Aug. 28, 2017. arXiv: [cs.LG/1708.07747](https://arxiv.org/abs/1708.07747) [cs.LG].
- [16] Ziqian Xie, Odelia Schwartz, and Abhishek Prasad. "Decoding of finger trajectory from ECoG using deep learning". In: *Journal of neural engineering* 15.3 (2018), p. 036009.
- [17] Lin Yao, Bingzhao Zhu, and Mahsa Shoaran. "Fast and accurate decoding of finger movements from ECoG through Riemannian features and modern machine learning techniques". In: *Journal of Neural Engineering* 19.1 (2022), p. 016037.

Anexos

Recomendaciones prácticas

- Al hacer la **cuantización**, si bien técnicamente no es obligatorio, es necesario utilizar un **dataset representativo** para que las salidas de la red cuantizada se encuentren en el rango en que deberían estar. Si uno no pone un dataset, o peor, si el dataset escogido no es representativo de las entradas reales, no habrá ningún error en el proceso de cuantización, pero la red entregará valores inesperados.
- Otra recomendación sobre la **cuantización**, es aplicarla **solamente a los pesos**, no a las funciones de activación de entradas ni salidas. Puede ser tentador cuantizar todo el modelo, pues cada entrada y salida utilizan por defecto 4 bytes cada una y en el caso de redes neuronales con muchas entradas, esta reducción a entradas de 1 byte podría ser significativa, pero cuantizarlas obliga a implementar la cuantización y de-cuantización de entradas y salidas de forma manual, pues la red totalmente cuantizada solo puede recibir y entregar valores de 8 bits, valores entre -128 y $+127$.
- Además de evitar problemas con la cuantización manual, otra razón para **cuantizar solo los pesos** de la red es que si se quiere pasar de la red cuantizada a la de precisión completa, basta con reemplazar el archivo del que se obtiene la red neuronal en la configuración del `.ioc`, manteniendo el nombre asignado a la red en STM32Cube.AI, simplemente se genera el código, y se puede construir y correr el proyecto con la nueva red sin tener que cambiar manualmente ninguna línea de código.
- En la Sección 5.4, en la parte en que se comentan los **buffers de entrada y salida**, de muestra que el atributo `shape.size` es 4 y `shape.data` es `{N,H,W,C}`. Si bien se menciona un par de ejemplos, los valores exactos de estas dimensiones puede ser algo confuso. La forma más sencilla de saber qué valores escribir en esta sección es utilizando un **network report**, esto debe hacerse después de crear la instancia de la red, pero antes de inicializarla, por ejemplo, se pueden agregar las siguientes líneas de código:

```
ai_network_report report;
if (ai_model_name_get_info(model_name, &report)){
    printf("Input shape (dims=%d): ", report.inputs->shape.size);
    for (int i = 0; i < report.inputs->shape.size; ++i)
        printf("%lu ", (unsigned long)report.inputs->shape.data[i]);
    printf("\r\n");
    printf("Output shape (dims=%d): ", report.outputs->shape.size);
    for (int i = 0; i < report.outputs->shape.size; ++i)
        printf("%lu ", (unsigned long)report.outputs->shape.data[i]);
    printf("\r\n");
}
```

- Para **cuantizar** con TensorFlow Lite, es necesario guardar el **modelo original**, uno no puede pasar desde un `.tflite` de precisión completa a un `.tflite` de 8 bits, ambos proceden directamente el modelo original, así que es buena idea guardarlo aunque uno crea que no lo va a usar después.